

Guia para o JupyterLab: Da Instalação à Análise de Dados para Planejamento Urbano e de Transportes

Sumário

1	Introdução: Uma Bancada de Trabalho Digital	2
1.1	Jupyter Notebook, JupyterLab	2
1.2	Por que o JupyterLab é a Ferramenta Recomendada?	2
2	Anaconda	3
2.1	Montando o Ambiente com a Distribuição Anaconda	3
2.2	Guia de Instalação Passo a Passo	4
2.2.1.1	Em Windows	4
2.2.1.8	Instalação e Conclusão	9
2.3	Instalação no macOS	11
2.4	Instalação no Linux	11
3	Criação de Ambiente Virtual e arquivos yaml.....	11
3.1	Criando e Usando Ambientes Virtuais	12
3.1.1.1	Criar um Ambiente do Zero (conda create).....	12
3.2	Arquivos environment.yaml.....	15
3.3	Gerenciamento Básico de Ambientes	17
3.3.1.1	Listar Ambientes:.....	17
3.3.1.2	Remover Ambientes.....	17
3.4	Recursos em Vídeo (Português)	17
4	4. Primeira Execução do JupyterLab.....	17
4.1	Iniciando o JupyterLab	17
4.2	Anatomia da Interface do JupyterLab	19
4.3	Criando seu Primeiro Notebook	20
4.4	Dominando o Notebook: Células, Comandos e Código.....	22
4.4.1.1	Os Blocos de Construção: Células de Código vs. Markdown	22
4.4.1.2	Os Dois Modos de Interação: Comando e Edição	25
4.5	Atalhos de Teclado	25
4.6	Salvando, Renomeando e Exportando Notebooks	26
5	Conclusão: Próximos Passos em sua Jornada com Dados	26

1 Introdução: Uma Bancada de Trabalho Digital

No campo do planejamento urbano e de transportes, a capacidade de analisar dados, testar hipóteses e comunicar resultados de forma clara é fundamental. Tradicionalmente, esse processo envolvia uma variedade de ferramentas desconectadas: planilhas para dados, software estatístico para análise, programas de GIS para visualização e editores de texto para relatórios. Os cadernos computacionais, ou *notebooks*, surgem como uma solução moderna e integrada para esse fluxo de trabalho. Um caderno computacional é um documento único que mescla código de programação executável, texto explicativo, equações matemáticas e visualizações ricas, como gráficos e mapas. Essa abordagem permite construir uma narrativa coesa em torno da análise, tornando-a ideal para exploração de dados, prototipagem rápida de modelos e comunicação transparente de resultados — atividades centrais para qualquer planejador.

1.1 Jupyter Notebook, JupyterLab

O Projeto Jupyter, uma iniciativa de código aberto, popularizou o uso de cadernos computacionais, começando com o **Jupyter Notebook** clássico. Esta ferramenta original oferece uma experiência simples e focada no documento, onde cada caderno é um universo próprio, aberto em sua própria aba do navegador. Embora eficaz, essa abordagem pode se tornar limitante para projetos complexos que exigem a manipulação de múltiplos arquivos, acesso ao terminal do sistema ou a visualização de dados brutos ao lado do código.

Reconhecendo essa necessidade, a comunidade de desenvolvedores lançou em 2018 o **JupyterLab**, a interface de próxima geração para o Projeto Jupyter. O JupyterLab não é uma ferramenta diferente, mas sim uma evolução que engloba e expande radicalmente as capacidades do Notebook clássico. Ele foi projetado desde o início para ser um ambiente de desenvolvimento interativo e completo.

1.2 Por que o JupyterLab é a Ferramenta Recomendada?

A recomendação de usar o JupyterLab hoje em dia baseia-se em suas vantagens claras para um fluxo de trabalho profissional e integrado. A transição do Notebook clássico para o JupyterLab pode ser comparada à mudança de trabalhar em uma única folha de mapa para utilizar um software de GIS completo. Enquanto o primeiro é excelente para apresentar um resultado final e estático, o segundo é uma bancada de trabalho dinâmica onde todas as ferramentas — o mapa, a tabela de atributos, os scripts de análise e o console — estão disponíveis em um único ambiente coeso.

As principais vantagens do JupyterLab incluem:

- **Ambiente Integrado (IDE-like):** Ao contrário do Jupyter Notebook, que isola cada arquivo, o JupyterLab oferece um ambiente de desenvolvimento integrado (IDE) dentro de uma única janela do navegador. Ele consolida cadernos, editores de texto, terminais de sistema, visualizadores de arquivos de dados e consoles de código interativos, criando um espaço de trabalho unificado.
- **Flexibilidade da Interface:** O layout do JupyterLab é totalmente personalizável. Os usuários podem arrastar, soltar e redimensionar abas para organizar seu espaço de trabalho da maneira que melhor se adapte à sua tarefa. É possível, por exemplo, ter um caderno de análise aberto ao lado de um arquivo CSV com os dados brutos, um mapa interativo e um terminal para executar comandos do sistema, tudo na mesma tela.

- **Suporte a Múltiplos Arquivos:** O JupyterLab gerencia nativamente uma vasta gama de tipos de arquivo, não se limitando aos cadernos (.ipynb). É possível abrir e editar scripts Python (.py), arquivos de texto, documentos Markdown (.md), e visualizar diretamente arquivos CSV, PDFs e imagens sem sair do ambiente.
- **Extensibilidade:** A arquitetura modular do JupyterLab foi projetada para ser estendida. Existe um ecossistema crescente de extensões que adicionam novas funcionalidades, como temas de interface (incluindo o popular modo escuro), depuradores de código visuais, integração com Git e muitas outras ferramentas.

A tabela a seguir resume as diferenças fundamentais entre as duas interfaces.

Característica	Jupyter Notebook (Clássico)	JupyterLab (Recomendado)
Interface	Focada em um único documento por aba	Integrada, com múltiplas abas e painéis em uma janela
Gerenciamento de Arquivos	Principalmente cadernos (.ipynb)	Suporte nativo a múltiplos tipos de arquivos (código, texto, dados)
Flexibilidade	Layout fixo	Layout flexível e personalizável (arrastar e soltar)
Ferramentas	Limitado ao caderno	Inclui terminal, console de código, editor de texto integrados
Extensibilidade	Limitada	Ecossistema robusto de extensões para novas funcionalidades
Experiência do Usuário	Simples, focada no documento	Completa, semelhante a um ambiente de desenvolvimento

2 Anaconda

2.1 Montando o Ambiente com a Distribuição Anaconda

Para começar a usar o JupyterLab, a maneira mais simples e robusta é através da **Distribuição Anaconda**. O Anaconda é mais do que apenas uma instalação do Python; é uma plataforma completa e gratuita voltada para ciência de dados. Ela vem com o Python, o gerenciador de pacotes e ambientes conda, e centenas das bibliotecas mais utilizadas em análise de dados, como Pandas, NumPy, Matplotlib e, claro, o próprio JupyterLab, tudo pré-instalado e testado para funcionar em conjunto.

Essa abordagem elimina a complexidade da configuração inicial, permitindo que o usuário se concentre na análise em vez de na compatibilização de dependências. O Anaconda fornece duas ferramentas principais para gerenciar o ambiente: o **Anaconda Navigator**, uma interface gráfica amigável, e o **Anaconda Prompt** (no Windows) ou o **Terminal** (no macOS/Linux), para quem prefere a [linha de comando](#).

2.2 Guia de Instalação Passo a Passo

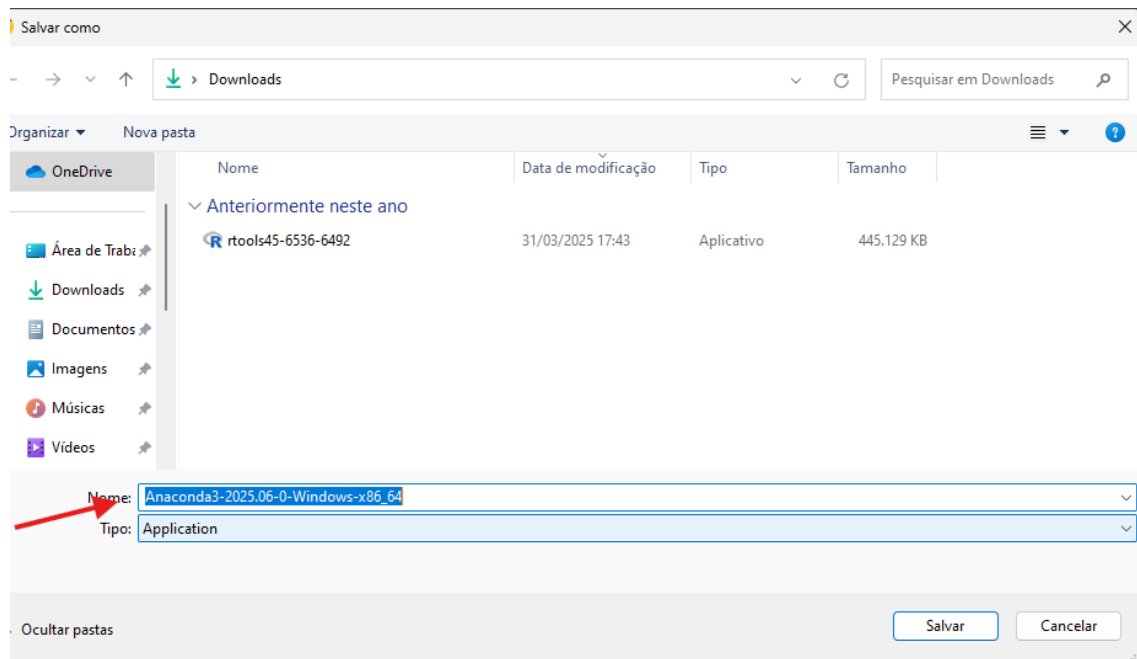
O primeiro passo é baixar o instalador apropriado para o seu sistema operacional a partir [do site oficial](#). Abaixo trazemos os passos por escritos e com prints, mas um bom recurso em vídeo pode ser encontrado [aqui](#).

2.2.1.1 Em Windows

2.2.1.2 Download

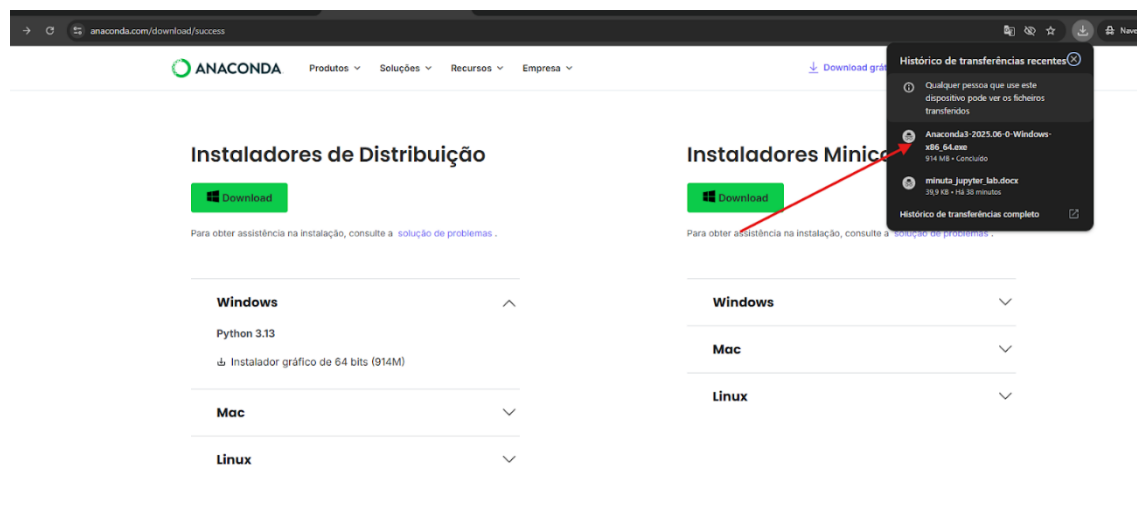
Acesse o link acima e baixe o instalador gráfico para Windows compatível com seu sistema operacional.

Instaladores de Distribuição



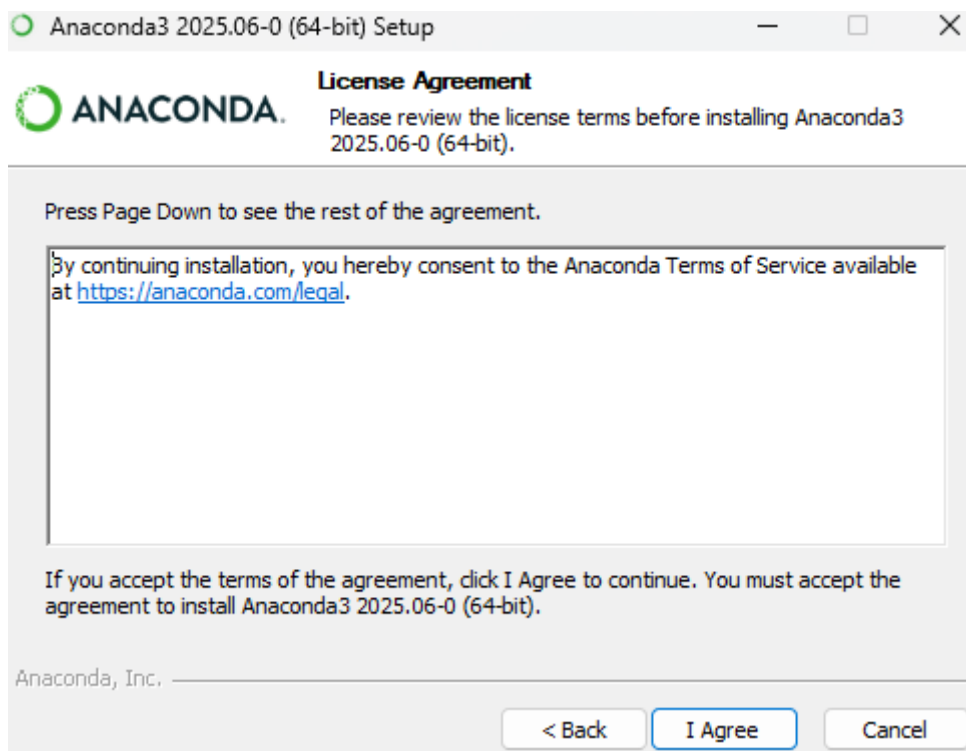
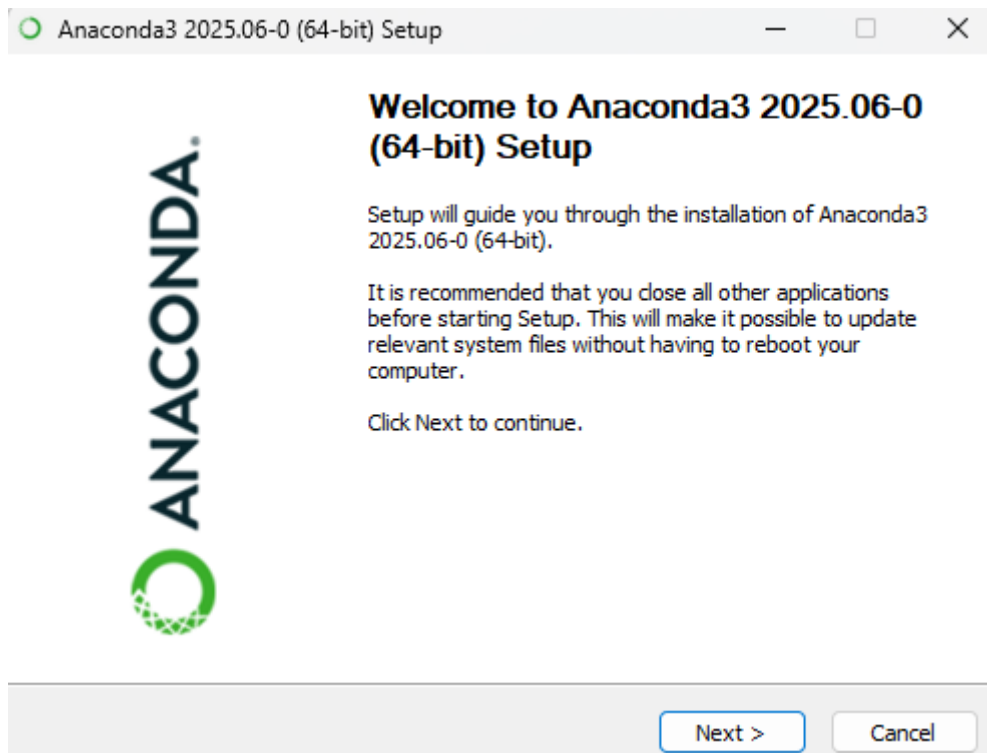
2.2.1.3 Executar o Instalador

Após o download, localize o arquivo.exe (por exemplo, Anaconda3-202X.XX-Windows-x86_64.exe) na sua pasta de Downloads e dê um duplo clique para iniciar a instalação.



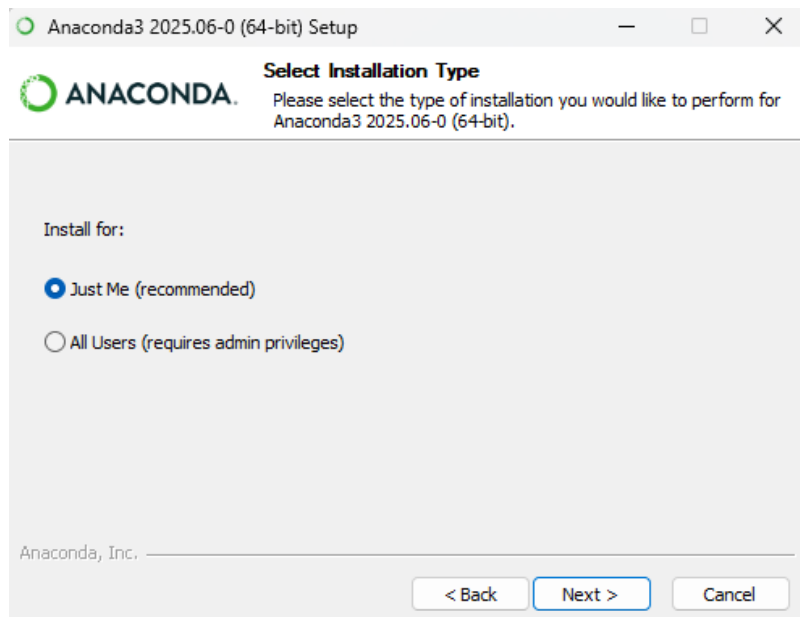
2.2.1.4 Telas Iniciais

O assistente de instalação será iniciado. Clique em "Next" na tela de boas-vindas. Em seguida, leia os termos da licença e clique em "I Agree" para continuar.



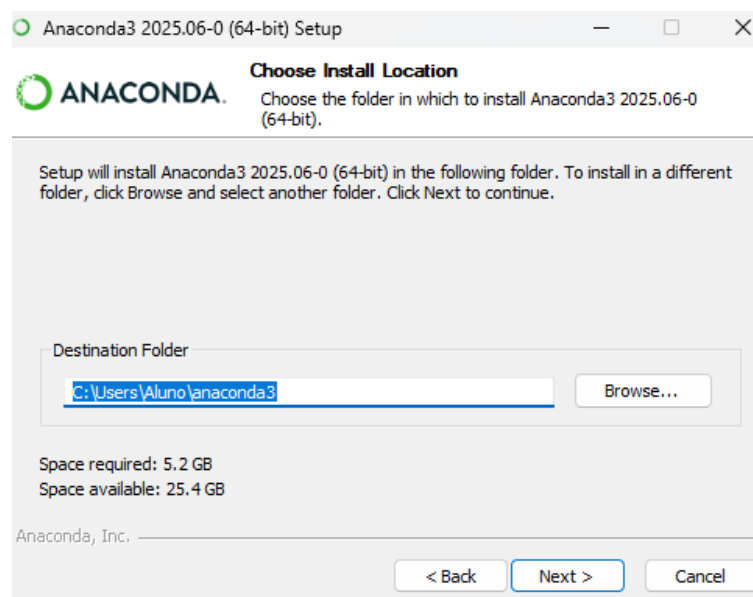
2.2.1.5 Tipo de Instalação

Na tela "Select Installation Type", a opção recomendada para a maioria dos usuários é "Just Me". Selecione-a e clique em "Next".



2.2.1.6 Local de Instalação

A próxima tela solicita o local de instalação. É fortemente recomendado manter o caminho padrão, que geralmente fica dentro do diretório do seu usuário (Ex.: C: \Users\SeuUsuario\anaconda3). Clique em "Next".



2.2.1.7 Opções Avançadas

Esta é a etapa mais importante da configuração.

Primeira opção: "Create shortcuts"

Cria atalhos na área de trabalho ou menu iniciar do computador. Opcional e depende do gosto de cada um, mas ajuda a localizar os softwares instalados.

Segunda opção: "Add Anaconda3 to my PATH environment variable"

O instalador não recomenda marcar esta caixa porque ao adicionar o Anaconda ao PATH durante a instalação, o instalador evita conflitos com outras instalações de Python que já estejam configuradas no sistema ou que venham a ser instaladas, **mas, como em tudo, há nuances:**

O que acontece, exatamente, se adicionar Anaconda ao PATH?

- O Windows passa a usar automaticamente o Python do Anaconda, sobrescrevendo outras versões do Python que possam estar instaladas (por exemplo, do Microsoft Store ou instaladas manualmente).
- Isso pode ser útil se você quer que *sempre* o Python padrão seja o do Anaconda, sem precisar ativar ambientes antes de usar.
- Mas pode gerar incompatibilidades se você trabalha em projetos que dependem de outra distribuição ou versão de Python.

O que acontece se NÃO adicionar ao PATH

- O Python do Anaconda só será acessível quando você abrir o **Anaconda Prompt** ou quando ativar manualmente o ambiente (conda activate) — mais detalhes sobre esses termos mais a frente.
- É mais seguro para quem tem múltiplas instalações de Python, porque evita substituir o comportamento global do sistema.
- Você ainda pode configurar o PATH manualmente no futuro, caso decida.

Em resumo:

- **Não adicionar ao PATH** → mais seguro, recomendado para quem já tem outro Python ou quer evitar conflitos.
- **Adicionar ao PATH** → prático se o Anaconda será sua única instalação de Python e você quer chamá-lo direto do terminal sem ativar nada.

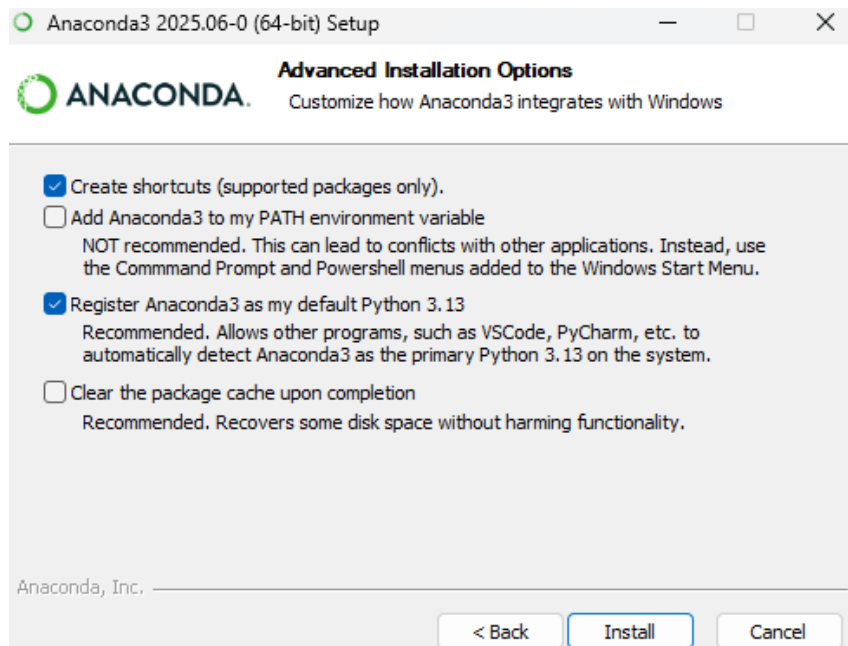
Dado o caráter do grupo de estudos, imaginamos que não haverá problemas em **adicionar** o Anaconda ao PATH e deixar a caixa marcada. Com avanço dos seus estudos em dados e em programação, caso essa alternativa se mostre desvantajosa, certamente você será capaz de resolver sem maiores problemas. Dado o perfil dos participantes do Grupo de Estudos, achamos razoável supor que é improvável que haja múltiplas instalações de Python ou cenários complexos de desenvolvimento que exijam isolamento.

Terceira opção: "Register Anaconda3 as my default Python 3.*"

Recomenda-se deixar esta opção marcada. Isso fará com que o Python instalado pelo Anaconda seja a versão padrão que outros programas que precisam de Python irão encontrar, o que é o comportamento desejado na maioria dos casos.

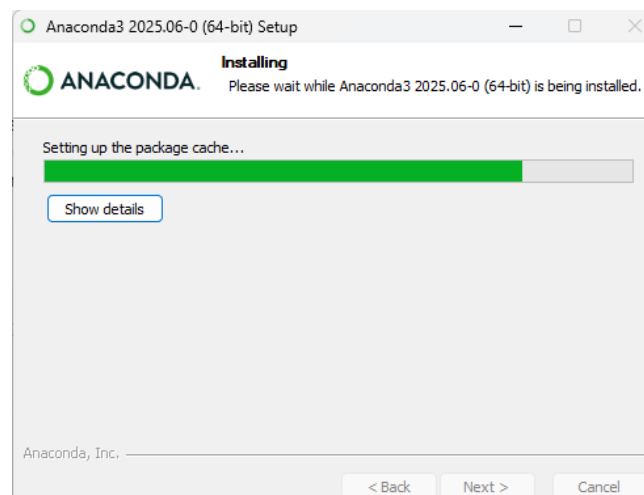
Quarta opção: "Clear the package cache upon completion"

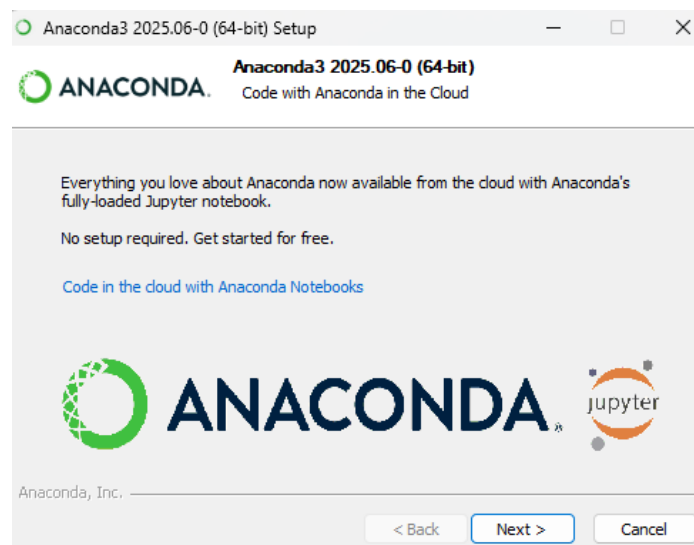
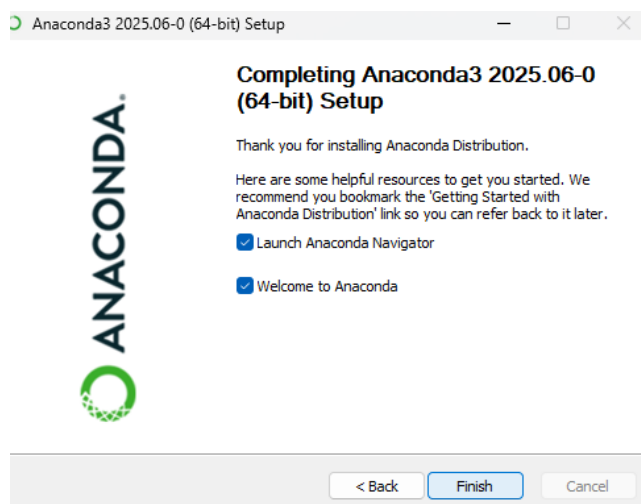
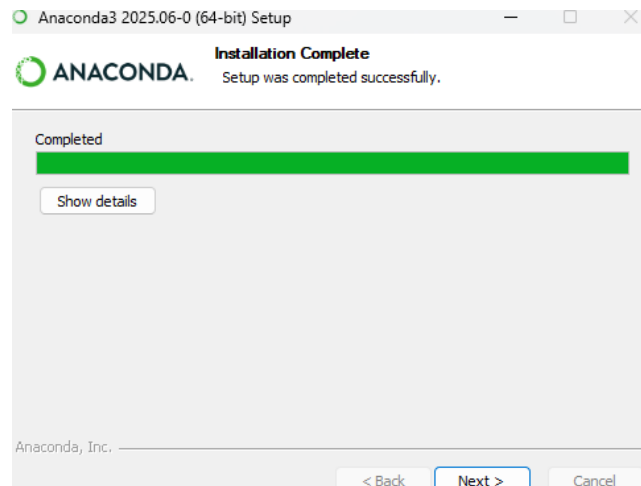
Limpa da memória do computador arquivos temporários da instalação. Recomendado por motivos evidentes: evitar desperdiçar espaço com arquivos agora já inúteis.



2.2.1.8 Instalação e Conclusão

Clique em "Install" para iniciar a cópia dos arquivos. O processo pode levar alguns minutos. Após a conclusão, clique em "Next" e, finalmente, em "Finish" para fechar o assistente.





2.3 Instalação no macOS

O processo no macOS é bastante semelhante, utilizando um instalador gráfico com a extensão.pkg:

1. Baixe o instalador gráfico para macOS do site oficial.
2. Dê um duplo clique no arquivo.pkg baixado, no local onde ele foi armazenado
3. Prossiga pelas telas do instalador, clicando em "Continue", aceitando os termos da licença ("Agree").
4. Selecione "Install for me only" e mantenha o local de instalação padrão. Clique em "Install". Pode ser necessário inserir a senha do seu usuário do Mac para autorizar a instalação.
5. Ao final do processo, clique em "Close" para fechar o instalador.

2.4 Instalação no Linux

No Linux, a instalação é feita através da linha de comando, o que oferece grande controle sobre o processo.

1. Download do Script: Abra o terminal e use um comando como wget ou curl para baixar o script de instalação do Anaconda. O link pode ser copiado da página de download do Anaconda.
2. Verifique a integridade do arquivo baixado usando o checksum SHA-256, comparando o resultado com o hash fornecido no site do Anaconda.
3. Execute o script de instalação usando o bash e siga as instruções.

3. Criação de Ambiente Virtual e arquivos yaml

Um dos recursos mais poderosos e essenciais do Anaconda é a capacidade de criar **ambientes virtuais**. Um ambiente virtual é um espaço de trabalho isolado e autocontido que possui sua própria instalação de Python e seu próprio conjunto de bibliotecas.

Essa contenção de bibliotecas em Python em módulos autocontidos é particularmente útil. Imagine, para fins de analogia, que você está trabalhando em dois projetos distintos simultaneamente: um "Plano Diretor de Drenagem Urbana" e uma "Análise de Viabilidade para uma nova linha de VLT".

- O projeto de drenagem requer um pacote de modelagem hidrológica (hidro-pacote v1.2) que, por sua vez, depende de uma versão específica e mais antiga de uma biblioteca de geoprocessamento (geolib v2.5).

- Já o projeto do VLT utiliza um software de simulação de tráfego de ponta (transito-sim v3.0) que exige a versão mais recente da mesma biblioteca (geolib v3.8).

Se você instalasse tudo em um único local, enfrentaria um problema: a instalação do transito-sim atualizaria a geolib para a versão 3.8, o que inevitavelmente quebraria o funcionamento do hidro-pacote que dependia da versão 2.5.

Os ambientes virtuais resolvem esse dilema. Eles funcionam como duas oficinas de trabalho completamente independentes. Na "Oficina Drenagem", você mantém a geolib v2.5 e todas as suas ferramentas associadas. Na "Oficina VLT", você instala a geolib v3.8 sem medo. Você pode entrar e sair de cada "oficina" (ativar e desativar os ambientes) sem que as ferramentas de uma interfiram na outra.

Essa prática traz três benefícios:

1. **Evita Conflitos de Dependência:** Garante que as bibliotecas de um projeto não interfiram com as de outro.
2. **Garante a Reprodutibilidade:** Permite que você (ou um colega) recrie o ambiente de software exato de um projeto no futuro, garantindo que o código continue funcionando e produzindo os mesmos resultados.
3. **Facilita o Gerenciamento:** Mantém seu sistema limpo e organizado, permitindo que você instale apenas o necessário para cada projeto e remova facilmente ambientes inteiros quando não são mais necessários.

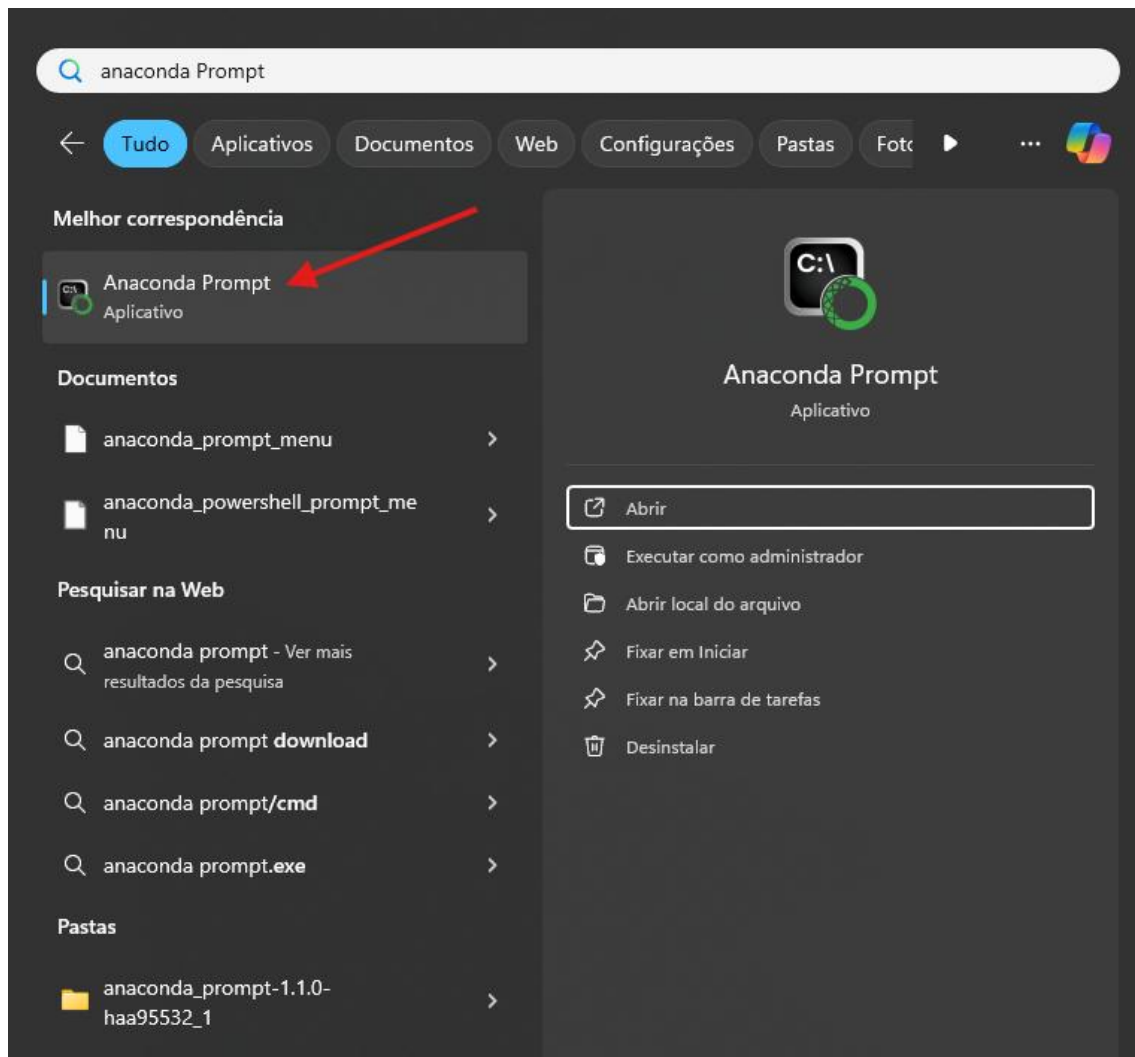
3.1 Criando e Usando Ambientes Virtuais

Todas as operações a seguir são realizadas no **Anaconda Prompt** (Windows) ou no **Terminal** (macOS/Linux). Mostraremos como criar um ambiente do zero, mas para os fins do grupo de estudos, mostraremos como fazer isso a partir de um arquivo .yaml, que nada mais é que um arquivo de texto contendo todas as instruções para instalação desde a versão do Python necessária, até as bibliotecas necessárias ao grupo de estudos, já nas versões "corretas", no sentido de serem versões que não conflitam umas com as outras.

3.1.1.1 Criar um Ambiente do Zero (conda create)

Se você usa Windows:

1. Clique no **Menu Iniciar**.
2. Digite Anaconda Prompt.
3. Clique no aplicativo que aparecer com esse nome.



Em seguida, digite cada comando abaixo na janela do **Anaconda Prompt** (ou Terminal) que você acabou de abrir e pressionando **Enter** ao final de cada um.

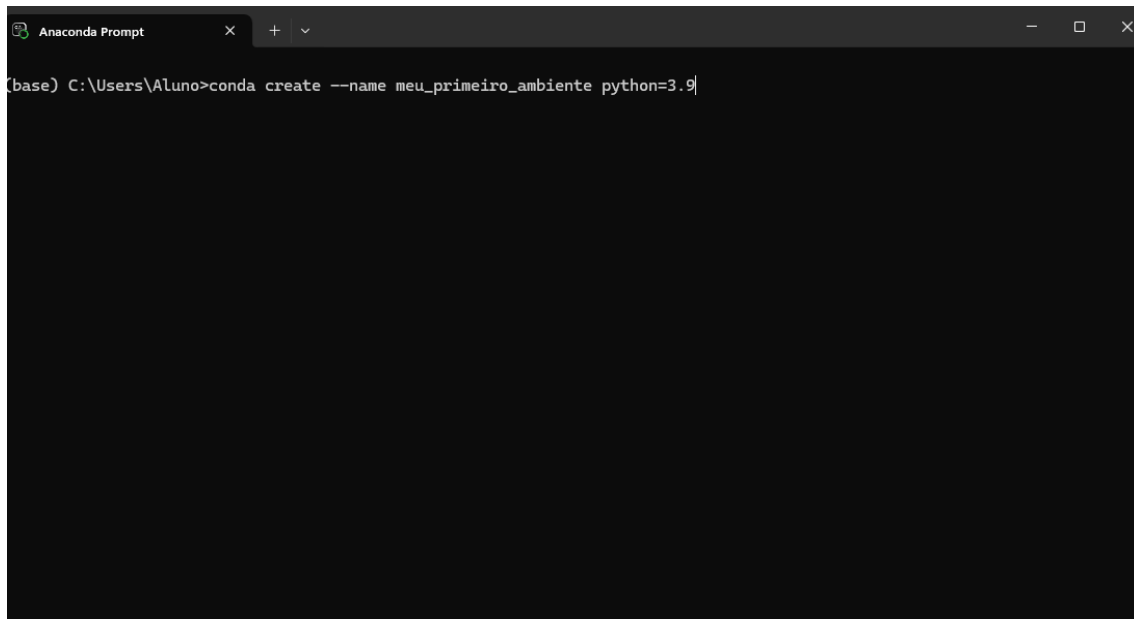
Passo 1: Criar o Ambiente Virtual (conda create)

Para criar uma nova "caixa de ferramentas", use o comando `conda create`. Você deve dar um nome ao ambiente e, opcionalmente, especificar a versão do Python e os pacotes que deseja instalar.

Digite o comando abaixo e pressione **Enter**:

```
conda create --name meu_primeiro_ambiente python=3.9
```

Neste comando, `create` é a ação, `--name` (ou a forma curta `-n`) faz o código entender que em seguida virá o nome do ambiente, no caso "meu_primeiro_ambiente", e `python=3.9` instala a versão 3.9 do Python nele. A versão do Python utilizada para o grupo de estudos é a 3.11, mas, como dito acima, não é necessário criar o ambiente do zero, estamos apenas mostrando os passos a fim de deixar este tutorial mais abrangente.



```

Anaconda Prompt
(base) C:\Users\Aluno>conda create --name meu_primeiro_ambiente python=3.9

```

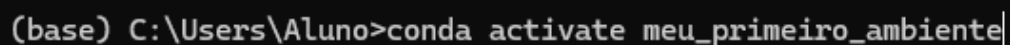
- **O que vai acontecer:** O Conda vai processar sua solicitação e, em seguida, mostrará uma lista de pacotes que ele precisa instalar para criar este ambiente. No final, ele fará uma pergunta para confirmar a operação: proceed ([y]/n)?
- **Sua Ação:** Digite a letra y e pressione Enter.
- **Resultado:** O Conda fará o download e a instalação dos pacotes. Pode levar um minuto. Quando terminar, você voltará para a linha de comando normal, ainda com o (base) no início.
-

Passo 2: Ativar o Ambiente (conda activate)

Para começar a usar sua nova "caixa de ferramentas", você precisa ativá-la.

```
conda activate meu_primeiro_ambiente
```

Após executar este comando, o prompt do seu terminal mudará, exibindo o nome do ambiente ativo entre parênteses, por exemplo: (meu_primeiro_ambiente) C: \Users\...>. Isso indica que você "entrou" no ambiente e qualquer comando Python ou conda que você executar agora acontecerá dentro deste espaço isolado.



```

(base) C:\Users\Aluno>conda activate meu_primeiro_ambiente

```

```
(base) C:\Users\Aluno>conda activate meu_primeiro_ambiente  
(meu_primeiro_ambiente) C:\Users\Aluno>
```

Passo 3: Instalar Pacotes em um Ambiente Ativo (conda install)

Com o ambiente ativo, você pode adicionar novas ferramentas a qualquer momento. Por exemplo:

```
conda install jupyterlab
```

Este comando buscará e instalará o jupyterlab e suas dependências exclusivamente dentro do ambiente meu_primeiro_ambiente, sem afetar o ambiente base ou quaisquer outros.

```
(meu_primeiro_ambiente) C:\Users\Aluno>conda install jupyterlab
```

```
Proceed ([y]/n)? y
```

```
Downloading and Extracting Packages:
```

```
Preparing transaction: done
```

```
Verifying transaction: done
```

```
Executing transaction: done
```

```
(meu_primeiro_ambiente) C:\Users\Aluno>
```

As operações que podem ser feitas no ambiente virtual serão detalhadas mais adiante.

Passo 4: Desativar o Ambiente (conda deactivate)

Quando terminar de trabalhar em um projeto, você pode "sair da caixa de ferramentas" e retornar ao ambiente base.

```
conda deactivate
```

O nome do ambiente desaparecerá do prompt do terminal, que voltará a exibir (base). Mas, você pode simplesmente fechar o prompt do anaconda.

3.2 Arquivos environment.yaml

Para garantir que um projeto seja verdadeiramente reprodutível, não basta apenas criar um ambiente; é preciso documentar sua configuração. O arquivo environment.yaml serve exatamente

a esse propósito. Ele é um arquivo de texto que funciona como uma "lista de materiais" ou um "blueprint" para o seu ambiente, permitindo que qualquer pessoa, em qualquer máquina, o recrie com exatidão.

Em um contexto profissional de planejamento, onde projetos são colaborativos e têm longa duração, o `environment.yaml` é um artefato de projeto tão crucial quanto o próprio caderno de análise ou os dados. Ele é a chave para a governança e a continuidade do trabalho. Nada mais é que um tipo de arquivo de texto com a estrutura abaixo.

```
name: samplenv
channels:
  - defaults
  - conda-forge
dependencies:
  - numpy=1.16.2
  - pandas=0.24.2
  - matplotlib=3.0.3
  - pillow=5.4.1
  - pip=19.0
  - plotly=3.7.0
  - scikit-learn=0.20.3
```

Em que:

- *name*: nome que o ambiente terá quando for criado;
- *channels*: fontes de onde o Conda deve baixar os pacotes. `conda-forge` é um canal comunitário com uma vasta coleção de pacotes; há também a fonte padrão (`defaults`) e muitas outras;
- *dependencies*: A lista de bibliotecas/pacotes a serem instalados. A subseção `pip` é usada para pacotes que não estão disponíveis nos canais Conda.

Se você receber um projeto que contém um arquivo `environment.yaml`, pode construir o ambiente completo com um único comando:

```
conda env create -f environment.yaml
```

O Conda lerá o arquivo, criará um novo ambiente com o nome especificado e instalará todas as dependências listadas. No entanto, ao se digitar apenas `environment.yaml`, o terminal/prompt assume que o arquivo de texto está no mesmo local no computador em que seu ambiente conda está instalado. Dessa forma, recomenda-se digitar o caminho completo de onde o arquivo `.yaml` está no seu computador, como, por exemplo,

```
C:\Users\josebezerra\Documentos\grupodeestudos\ambiente_roda.yaml
```

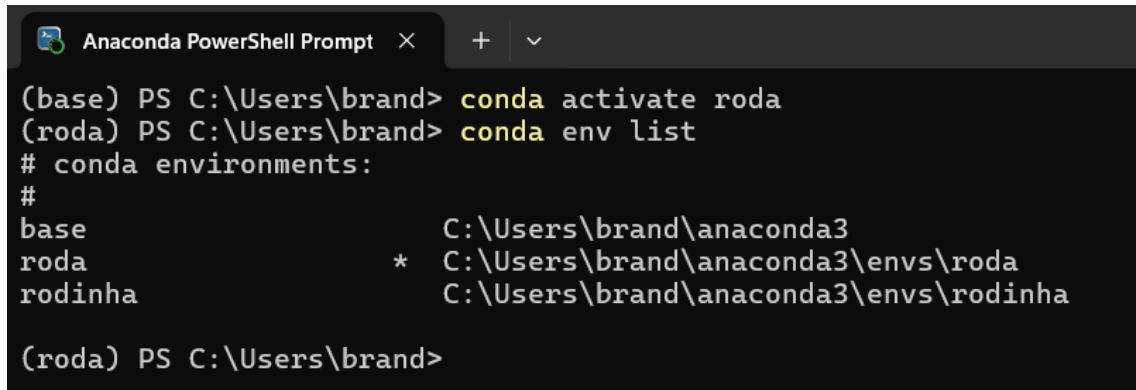

3.3 Gerenciamento Básico de Ambientes

3.3.1.1 Listar Ambientes:

Para ver todas as "caixas de ferramentas" que você criou, use o comando:

```
conda env list
```

Isso exibirá uma lista de todos os seus ambientes, com um asterisco (*) marcando o que está ativo no momento.



```
Anaconda PowerShell Prompt x + v
(base) PS C:\Users\brand> conda activate roda
(roda) PS C:\Users\brand> conda env list
# conda environments:
#
base                C:\Users\brand\anaconda3
roda                * C:\Users\brand\anaconda3\envs\roda
rodinha             C:\Users\brand\anaconda3\envs\rodinha

(roda) PS C:\Users\brand>
```

3.3.1.2 Remover Ambientes

Para excluir um ambiente que não é mais necessário e liberar espaço em disco, use o comando:

```
conda env remove --name meu_primeiro_ambiente
```

Este comando remove permanentemente o ambiente e todos os pacotes instalados nele.

3.4 Recursos em Vídeo (Português)

Como criar um ambiente virtual no anaconda? - YouTube, accessed August 11, 2025,
<https://www.youtube.com/watch?v=K-9VLQum6z0>

#15 ANACONDA CRIAÇÃO DE AMBIENTES VIRTUAIS - YouTube, accessed August 11, 2025,
<https://www.youtube.com/watch?v=K30bFGwyocY>

Tutorial de introdução ao JupyterLab: Instalação, extensões e muito mais | DataCamp, accessed August 11, 2025, <https://www.datacamp.com/pt/tutorial/installing-jupyter-notebook>

4 Primeira Execução do JupyterLab

Com o Anaconda instalado e os ambientes, também, é hora de iniciar o JupyterLab e explorar sua interface.

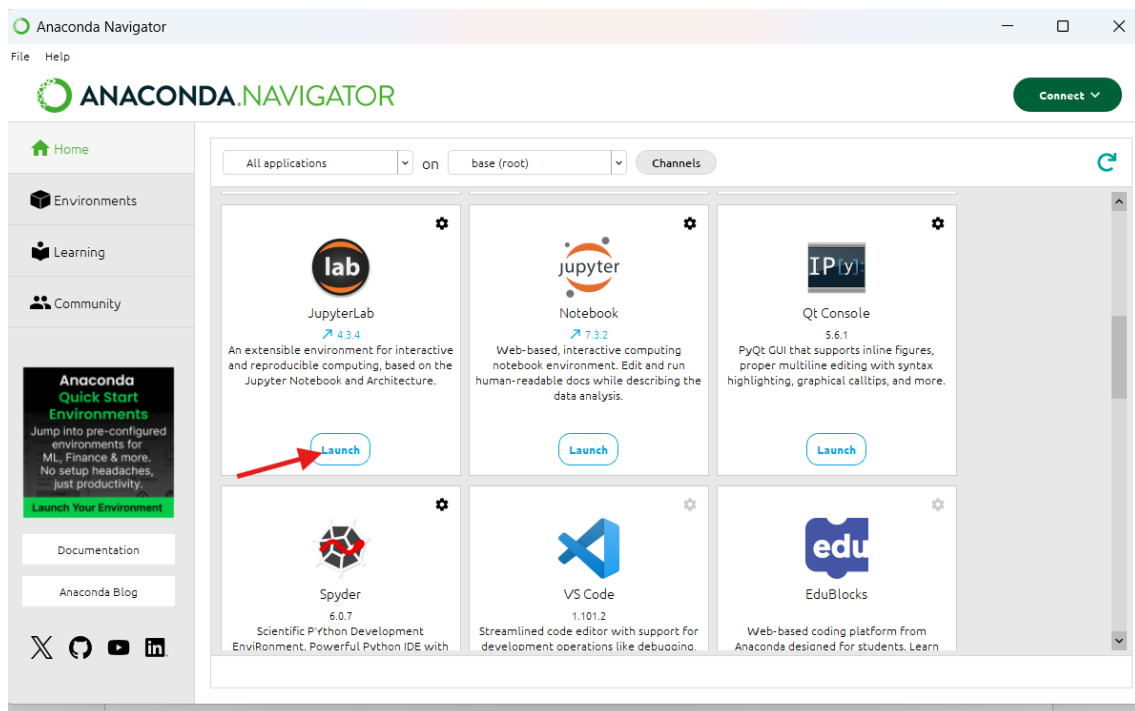
4.1 Iniciando o JupyterLab

Existem duas maneiras principais de iniciar a aplicação.

Método 1: Anaconda Navigator (Interface Gráfica)

Este é o método mais simples para iniciantes.

1. Abra o **Anaconda Navigator** (pode ser encontrado no menu Iniciar do Windows ou na pasta de Aplicativos do macOS).
2. Na tela principal ("Home"), você verá uma série de aplicações disponíveis. Localize o card do **JupyterLab**.
3. Clique no botão **"Launch"** abaixo do ícone do JupyterLab. Uma nova aba será aberta no seu navegador padrão com a interface do JupyterLab.



Método 2: Linha de Comando (Anaconda Prompt/Terminal)

```
Anaconda PowerShell Prompt
(base) PS C:\Users\brand> conda activate roda
(roda) PS C:\Users\brand> conda env list
# conda environments:
#
base                  C:\Users\brand\anaconda3
roda                  * C:\Users\brand\anaconda3\envs\roda
rodinha              C:\Users\brand\anaconda3\envs\rodinha

(roda) PS C:\Users\brand> jupyter lab
```

Isso iniciará o servidor do JupyterLab e abrirá a interface no seu navegador de internet padrão.

O prompt, por sua vez, ficará com o seguinte aspecto:

```
Anaconda PowerShell Prompt
roda
rodinha
C:\Users\brand\anaconda3\envs\roda
C:\Users\brand\anaconda3\envs\rodinha

(roda) PS C:\Users\brand> jupyter lab
[2025-08-14 01:16:13.089 ServerApp] jupyter_lsp | extension was successfully linked.
[2025-08-14 01:16:13.098 ServerApp] jupyter_server_terminals | extension was successfully linked.
[2025-08-14 01:16:13.108 ServerApp] jupyterlab | extension was successfully linked.
[2025-08-14 01:16:13.858 ServerApp] notebook_shim | extension was successfully linked.
[2025-08-14 01:16:13.953 ServerApp] notebook_shim | extension was successfully loaded.
[2025-08-14 01:16:13.956 ServerApp] jupyter_lsp | extension was successfully loaded.
[2025-08-14 01:16:13.957 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[2025-08-14 01:16:13.969 LabApp] JupyterLab extension loaded from C:\Users\brand\anaconda3\envs\roda\Lib\site-packages
\jupyterlab
[2025-08-14 01:16:13.969 LabApp] JupyterLab application directory is C:\Users\brand\anaconda3\envs\roda\share\jupyter\
lab
[2025-08-14 01:16:13.971 LabApp] Extension Manager is 'pypi'.
[2025-08-14 01:16:14.001 ServerApp] jupyterlab | extension was successfully loaded.
[2025-08-14 01:16:14.002 ServerApp] Serving notebooks from local directory: C:\Users\brand
[2025-08-14 01:16:14.002 ServerApp] Jupyter Server 2.16.0 is running at:
[2025-08-14 01:16:14.002 ServerApp] http://localhost:8888/lab?token=e503dffbe5ccce72ab9e575d443e7fd6a7b9da6c5e481090
[2025-08-14 01:16:14.003 ServerApp] http://127.0.0.1:8888/lab?token=e503dffbe5ccce72ab9e575d443e7fd6a7b9da6c5e481090
[2025-08-14 01:16:14.003 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirm
ation).
[C 2025-08-14 01:16:14.088 ServerApp]

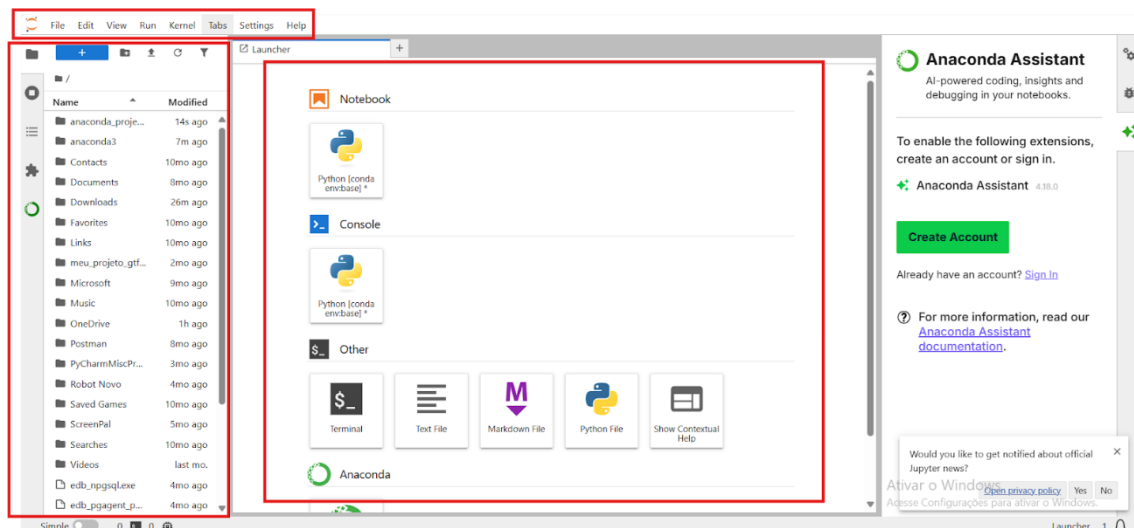
To access the server, open this file in a browser:
file:///C:/Users/brand/AppData/Roaming/jupyter/runtime/jpserver-19760-open.html
Or copy and paste one of these URLs:
http://localhost:8888/lab?token=e503dffbe5ccce72ab9e575d443e7fd6a7b9da6c5e481090
```

Essa janela é o motor por trás da aplicação. Ele exibe logs do servidor e **precisa permanecer aberto durante toda a sua sessão de trabalho**. Se você fechar essa janela do terminal, o servidor será desligado e o JupyterLab deixará de funcionar.

4.2 Anatomia da Interface do JupyterLab

Ao abrir o JupyterLab, você será recebido por uma interface rica e organizada.

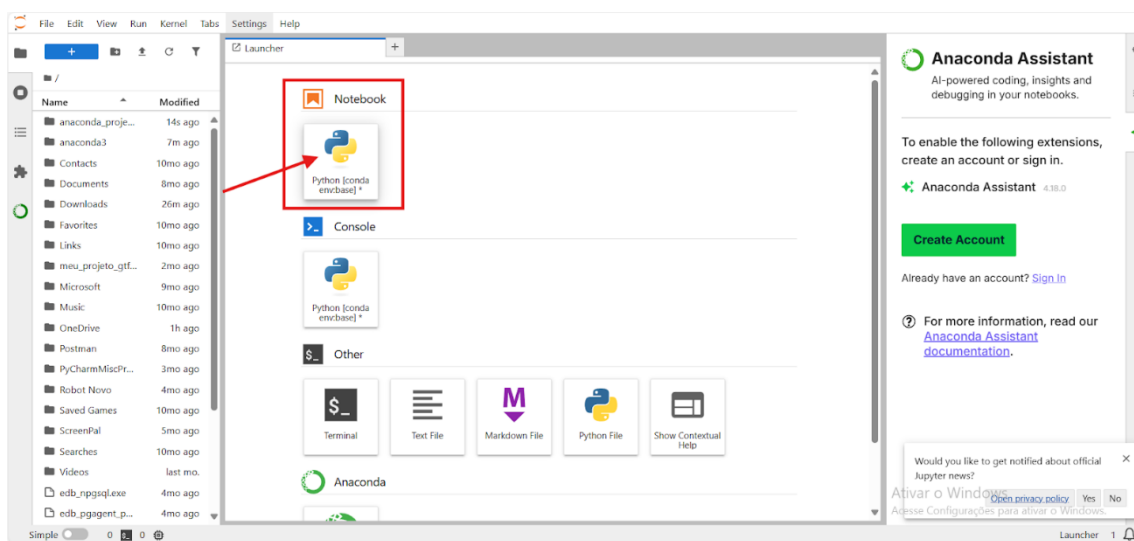
- **Barra de Menus (Topo):** Contém menus familiares como File (Arquivo), Edit (Editar), View (Visualizar), Run (Executar), Kernel, Tabs (Abas), Settings (Configurações) e Help (Ajuda). A maioria das ações globais pode ser encontrada aqui.
- **Barra Lateral Esquerda (Recolhível):** Um painel vertical que dá acesso rápido a várias ferramentas essenciais. Clicar em um ícone abre a aba correspondente.
 - **Explorador de Arquivos:** Permite navegar, criar, renomear e excluir arquivos e pastas no diretório do seu projeto. **Navegue para o local onde a pasta dos scripts do RODA está instalada**, à semelhança do que é feito no explorador de arquivos do Windows.
 - **Kernels e Terminais Ativos:** Lista todos os processos de computação (kernels) e terminais que estão em execução em segundo plano. É aqui que você pode desligar um kernel para liberar recursos.
 - **Tabela de Conteúdos:** Gera automaticamente um índice navegável do seu caderno com base nos títulos e subtítulos que você cria em células de texto (Markdown).
 - **Gerenciador de Extensões:** Permite pesquisar e instalar extensões para personalizar e adicionar funcionalidades ao JupyterLab.
- **Área de Trabalho Principal:** É o espaço central e flexível onde você trabalha. Cada arquivo ou atividade que você abre (um caderno, um arquivo de texto, um terminal) aparece em sua própria **aba**. Você pode arrastar essas abas para reorganizar a área de trabalho, criando painéis lado a lado ou empilhados para visualizar múltiplos documentos ao mesmo tempo. A aba ativa é destacada com uma borda colorida no topo.

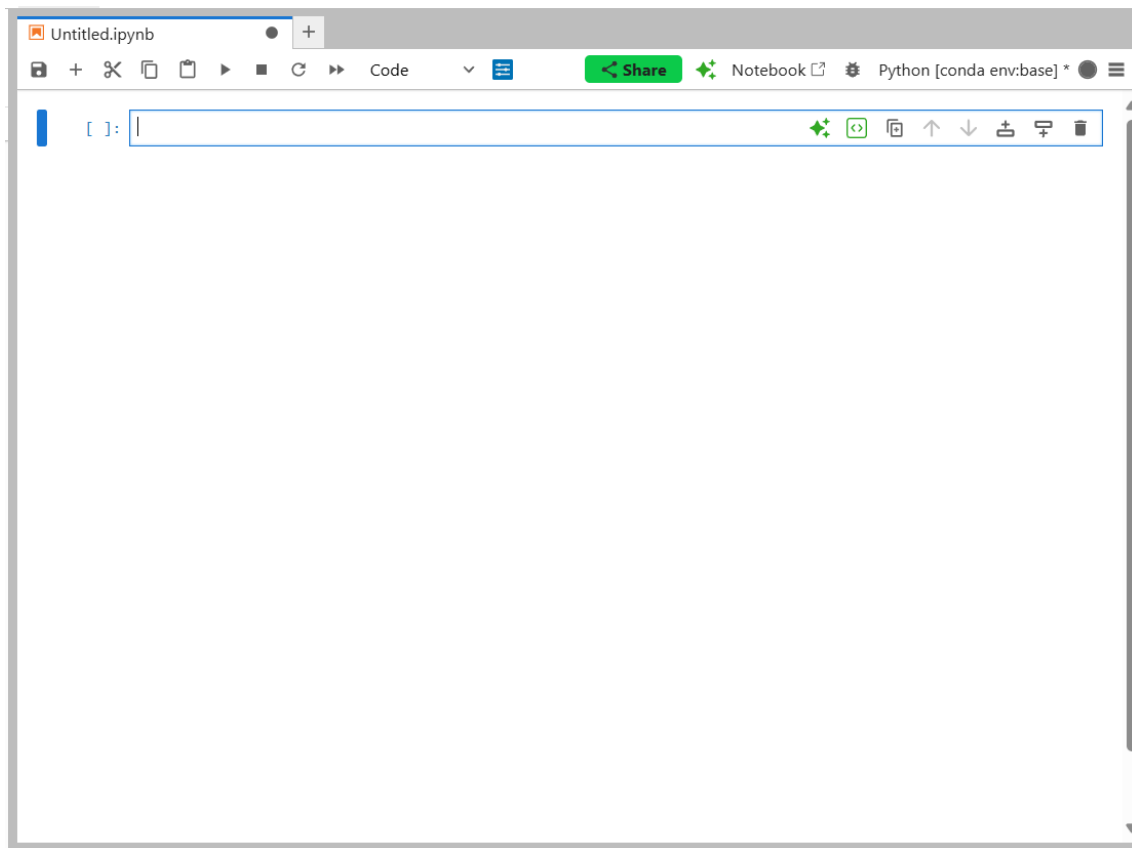


4.3 Criando seu Primeiro Notebook

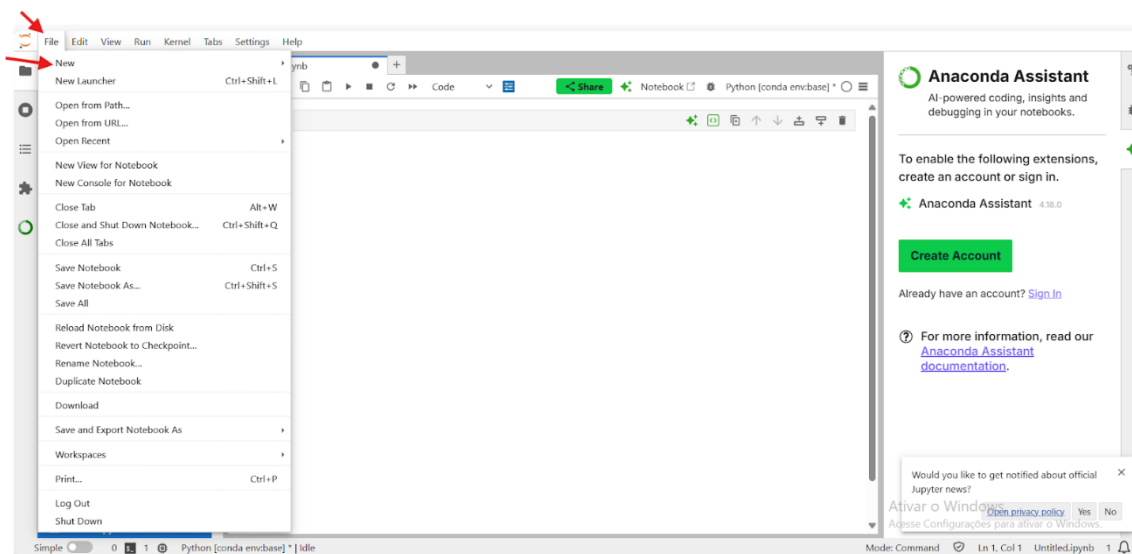
Para começar a análise, você precisa criar um caderno.

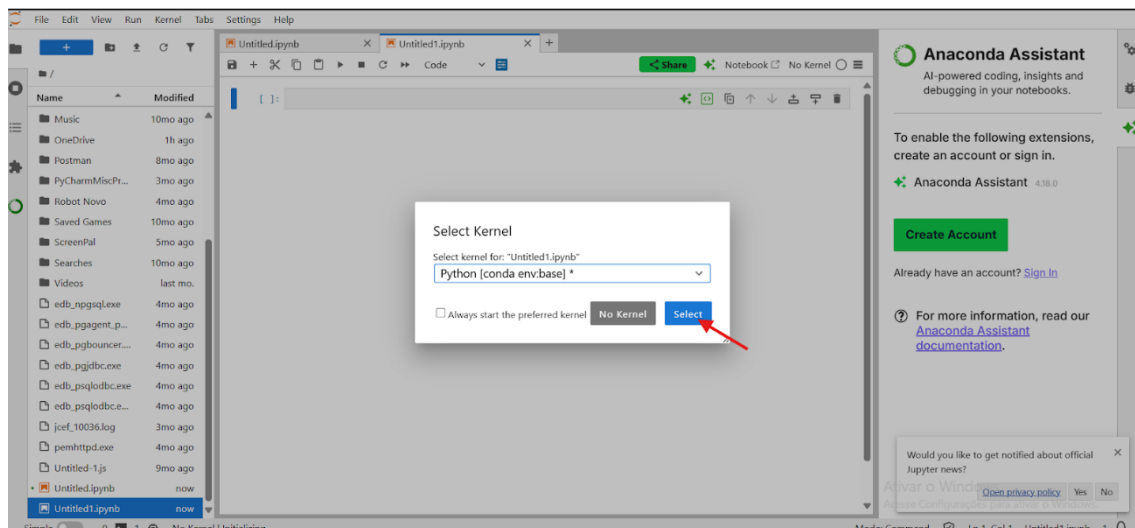
- **Método 1 (Launcher):** Na área de trabalho principal, a aba "Launcher" geralmente está aberta por padrão. Na seção "Notebook", clique no ícone **"Python 3 (ipykernel)"**. Isso criará e abrirá um novo caderno chamado Untitled.ipynb.





- **Método 2 (Menu):** Vá para a barra de menus e selecione File > New > Notebook. Uma caixa de diálogo pedirá para você selecionar o kernel (o motor computacional). Escolha "Python 3 (ipykernel)" e clique em "Select".





4.4 Dominando o Notebook: Células, Comandos e Código

O coração de um caderno Jupyter são suas células. Elas são os blocos de construção onde você escreve e executa código, adiciona texto explicativo e gera visualizações.

4.4.1.1 Os Blocos de Construção: Células de Código vs. Markdown

No Jupyter Notebook, existem dois tipos principais de células utilizados com frequência. A primeira delas é a **célula de Markdown**, utilizada para inserir texto formatado por meio da sintaxe Markdown, uma linguagem de marcação leve e intuitiva. Ao executar uma célula de Markdown, o conteúdo é renderizado como texto com formatação, permitindo criar:

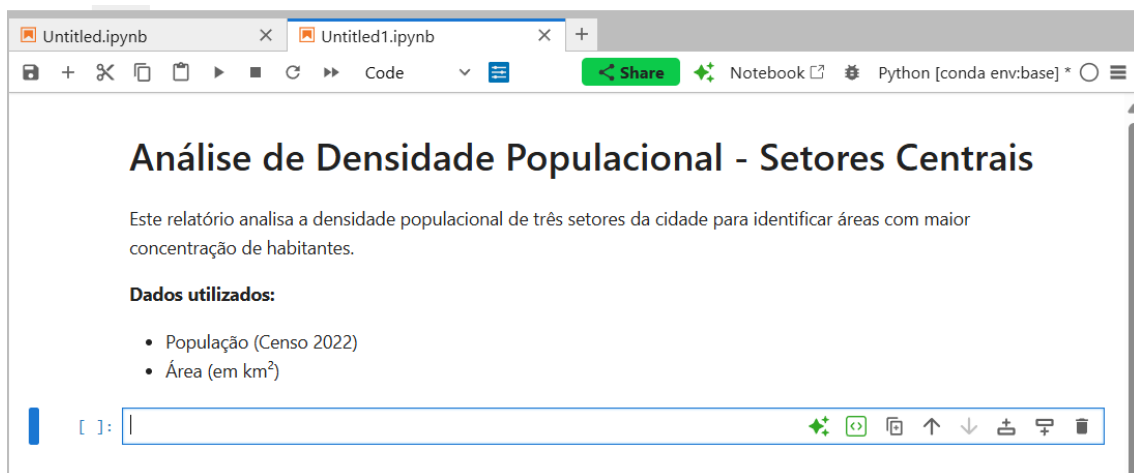
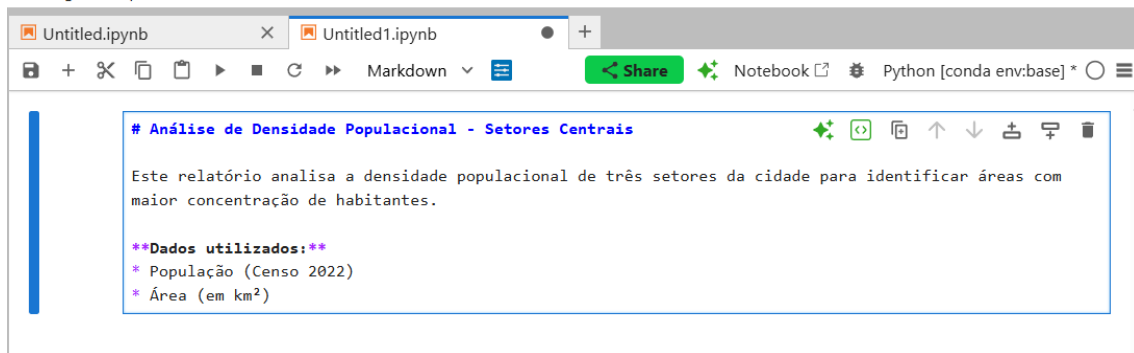
- cabeçalhos;
- listas;
- formatações em negrito, itálico etc.;
- hiperlinks;
- blocos de código e citações; etc.

Essa célula é ideal para documentar a análise de forma clara, estruturada e visualmente organizada.

Passo 1: Criando um Título com uma Célula de Markdown

Queremos começar com um título e uma breve descrição. Para isso, usamos uma célula de Markdown.

1. Selecione a Célula: Clique na primeira célula.
2. Mude para Markdown: Pressione a tecla M. A indicação `In [ ]:` do lado esquerdo vai desaparecer.
3. Comece a Escrever: clique duplamente na célula para entrar no Modo de Edição.
4. Digite o seguinte texto:
5. Renderize o Texto: Pressione **Shift + Enter** (teste também **Ctrl + Enter** e note a diferença). A célula será "executada", e o texto aparecerá formatado.

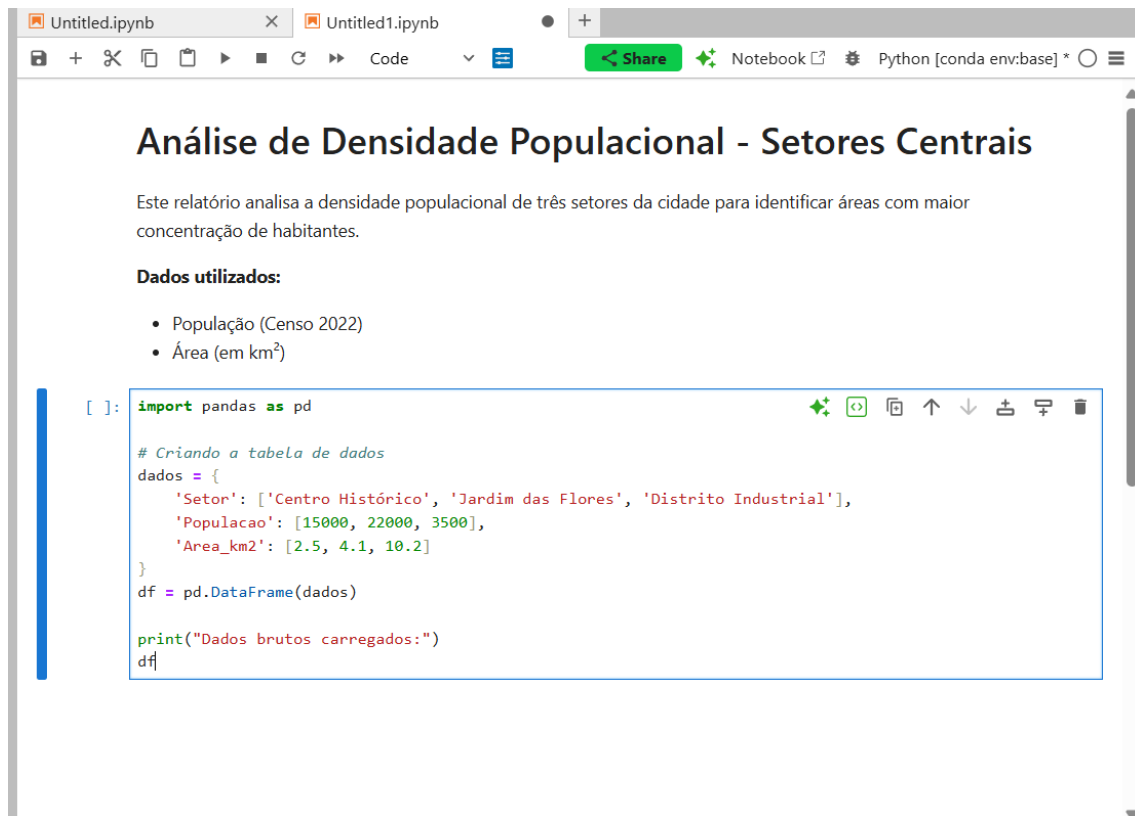


Posto isso, há as células de código, que se destinam a conter código executável (Python, em nosso contexto). Ao executar uma célula de código, o Python a processa e exibe qualquer saída (texto, tabelas, gráficos) logo abaixo dela. As células de código são identificadas por um marcador `In [ ]:` à sua esquerda.

Passo 1: Inserindo e Executando uma Célula de Código

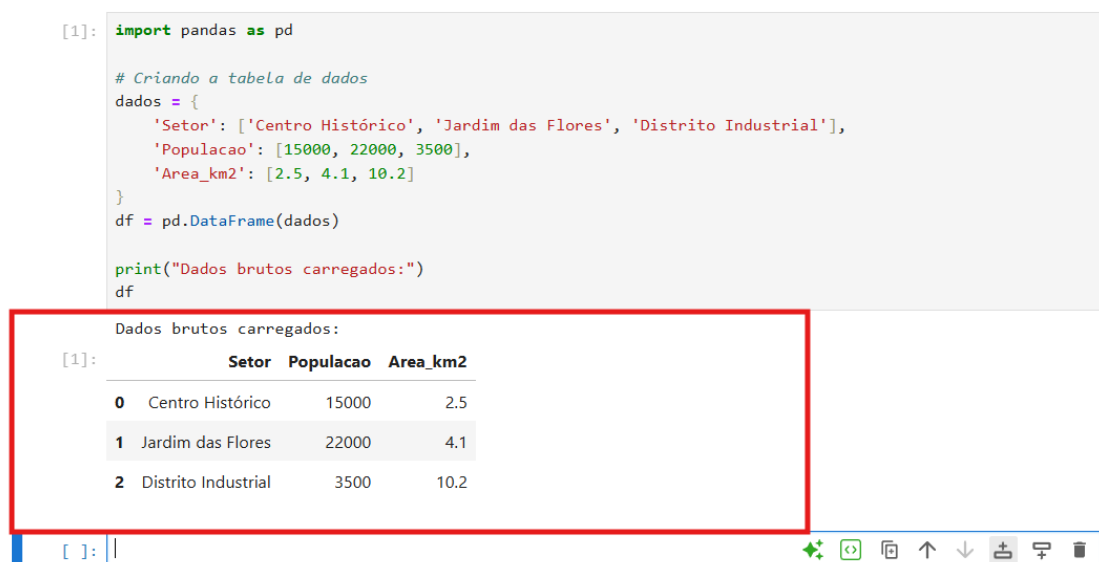
Agora, vamos usar uma célula de **Código** para importar a biblioteca pandas e criar nossos dados.

1. Selecione a Nova Célula: Ela já deve estar selecionada e no Modo de Edição (borda verde). Se não estiver, clique nela e pressione Enter.
2. Digite o seguinte código Python:



3. Execute o Código: Pressione Shift + Enter. O Jupyter executará o código e mostrará a saída (o resultado do print e a tabela df) logo abaixo.

A célula e sua saída ficarão assim:



4.4.1.2 Os Dois Modos de Interação: Comando e Edição

Compreender os dois modos de interação com as células é crucial para usar o JupyterLab de forma eficiente. A borda da célula ativa muda de cor para indicar o modo atual.

- **Modo de Comando:** Neste modo, suas teclas executam comandos no nível do caderno, como navegar entre células, criar novas células, ou copiar e colar células inteiras. Você não pode digitar texto dentro da célula. Para entrar no Modo de Comando, pressione a tecla Esc ou clique na margem esquerda da célula, fora da área de texto.
- **Modo de Edição:** Este modo permite que você digite e edite o conteúdo (código ou texto) dentro da célula. Um cursor de texto piscará dentro da célula. Para entrar no Modo de Edição, pressione a tecla Enter ou clique com o mouse dentro da área de texto da célula.

4.5 Atalhos de Teclado

A verdadeira fluidez no JupyterLab vem do uso de atalhos de teclado, especialmente no Modo de Comando. Eles permitem que você manipule seu caderno sem tirar as mãos do teclado, acelerando drasticamente o fluxo de trabalho.

Ação	Atalho (Modo de Comando)	Descrição
Executar Célula	Shift + Enter	Executa a célula atual e seleciona a próxima.
	Ctrl + Enter	Executa a célula atual e mantém a seleção nela.
	Alt + Enter	Executa a célula atual e insere uma nova célula abaixo.
Mudar Tipo de Célula	M	Converte a célula para Markdown .
	Y	Converte a célula para código (P ython).
Manipular Células	A	Insere uma nova célula A cima (Above).
	B	Insere uma nova célula a Baixo (Below).
	X	Recorta (cut) a célula selecionada.
	C	C opia a célula selecionada.
	V	Cola (paste) a célula recortada/copiada abaixo.
	Shift + V	Cola (paste) a célula recortada/copiada acima.
	D, D (D duas vezes)	D eleta a célula selecionada.
	Z	Desfaz (undo) a última exclusão de célula.
	S	S alva o caderno (embora haja “salvamentos” automáticos periódicos).

4.6 Salvando, Renomeando e Exportando Notebooks

Todas essas ações fundamentais são acessíveis através do menu File na barra superior.

- **Salvar:** Para salvar o progresso do seu caderno, você pode usar o menu File > Save Notebook, clicar no ícone de disquete na barra de ferramentas do caderno, ou usar o atalho de teclado Ctrl + S (ou simplesmente S no Modo de Comando).⁷⁰
- **Renomear:** O nome padrão de um novo caderno é Untitled.ipynb. Para renomeá-lo, você pode clicar com o botão direito do mouse sobre o nome do arquivo no Explorador de Arquivos (na barra lateral esquerda) e selecionar "Rename", ou, com o caderno aberto, ir em File > Rename Notebook....
- **Exportar:** A capacidade de exportar seu caderno para diferentes formatos é essencial para a comunicação de resultados. A opção se encontra em File > Export Notebook As....⁷² Cada formato atende a uma necessidade profissional específica:
 - **Exportar para HTML (.html):** Cria uma página da web autônoma que pode ser aberta em qualquer navegador. É a melhor opção para compartilhar uma análise completa e interativa (se contiver gráficos dinâmicos) com stakeholders que não possuem Python ou Jupyter. O relatório pode ser facilmente enviado por e-mail ou hospedado em um site.⁷²
 - **Exportar para PDF (.pdf):** Gera um documento estático com aparência profissional, ideal para relatórios formais, documentação de projetos ou submissões acadêmicas. A conversão para PDF geralmente requer a instalação de dependências adicionais (como uma distribuição LaTeX), mas não é necessário se preocupar com isso nesse momento.
 - **Exportar para Script Python (.py):** Extrai apenas o conteúdo das células de código do seu caderno e o salva como um script Python padrão. As repercussões disso são importantes, mas além do escopo deste tutorial..

5 Conclusão: Próximos Passos em sua Jornada com Dados

Este guia cobriu os passos mais básicos e essenciais para transformar o JupyterLab em sua principal bancada de trabalho para análise de dados em planejamento urbano e de transportes. Desde a instalação com Anaconda e a organização de projetos com ambientes virtuais, passando pela navegação fluida na interface e o domínio das células e comandos. A verdadeira maestria, no entanto, vem com a prática.

Por fim, há um vasto arcabouço de vídeos e sites que tratam dos primeiros passos postos aqui. Tanto para esse grupo de estudos quanto para o resto de sua jornada com análise de dados, seja em Python, seja com outra linguagem, incentivamos fortemente que vocês façam pesquisas conteúdos por si mesmos. Parte da vantagem do Python é que há uma grande comunidade ativa divulgando conhecimento e a consulta a esse tipo de conteúdo é um hábito que sempre será necessário, pois não há como cobrir todos os detalhes desse campo do conhecimento com apenas um material. Dúvidas sempre irão surgir, ou a memória de algo precisará ser refrescada eventualmente. Além disso, dados os rápidos avanços, sempre é necessário caminhar na direção de se manter atualizado.

